22

超越物理内存：策略

在虚拟内存（virtual memory）管理器里，空闲内存多的时候日子很好过。发生缺页异常（page fault）时，从空闲页链表（free-page list）里找一个空闲页，分给发生缺页的那一页就行了。嘿，操作系统，恭喜！你又成功了。

可惜，空闲内存少的时候，事情就变得有点意思了。这时，**内存压力**（memory pressure）会迫使操作系统开始把页（page）**换出**（paging out），给正在使用的页腾出地方。决定**淘汰**（evict）哪一页（或哪些页），是操作系统**置换策略**（replacement policy）的职责。从历史上看，这是早期虚拟内存系统要做的最重要的决策之一，因为当年的系统物理内存（physical memory）很少。至少，这组策略本身也挺有意思，值得多了解一些。于是，我们要解决的问题是：

关键问题：如何决定淘汰哪一页

操作系统如何决定把哪一页（或哪些页）从内存中淘汰出去？这个决定由系统的置换策略（replacement policy）做出，置换策略通常遵循一些通用原则（下面会讨论），但也会加入一些调整，以避免在极端情况下出现异常行为。

22.1 缓存管理

在深入讨论各种策略之前，我们先更详细地描述一下要解决的问题。既然主存保存的是系统中全部页的一个子集，那么完全可以把它看作系统中虚拟内存页的**缓存**（cache）。因此，为这个缓存选择置换策略（replacement policy）时，我们的目标是让**缓存未命中**（cache miss）的次数最少，也就是让必须从磁盘取页的次数最少。换个角度，也可以说我们的目标是让**缓存命中**（cache hit）的次数最多，也就是让被访问的页能在内存中找到的次数最多。

知道了缓存命中和未命中的次数，就能算出程序的**平均内存访问时间**（average memory access time，**AMAT**），计算机体系结构的设计者就用这个指标来衡量硬件缓存[HP06]。具体来说，有了这些值，就可以按下式算出程序的 AMAT：

$$\text{AMAT} = T_M + (P_{\text{Miss}} \cdot T_D) \tag{22.1}$$

其中 T_M 是访问内存的开销，T_D 是访问磁盘的开销，P_{Miss} 是在缓存中找不到数据（未命中）的概率；P_{Miss} 的取值在 0.0 到 1.0 之间，有时我们也用百分比形式的未命中率（miss

rate）来代替概率（例如，未命中率为 10% 就表示 $P_{\text{Miss}} = 0.10$）。注意，访问内存中数据的开销你总是要付的；但一旦未命中，你还得额外付出从磁盘取数据的开销。

举个例子，假设有一台机器，地址空间（address space）很小，只有 4 KB，页大小（page size）为 256 字节。于是，虚拟地址（virtual address）由两部分组成：4 位的 VPN（最高几位）和 8 位的偏移量（offset，最低几位）。因此，这个例子中的进程（process）总共可以访问 2^4 即 16 个虚拟页（virtual page）。假设进程产生了以下内存引用，也就是以下虚拟地址：0x000、0x100、0x200、0x300、0x400、0x500、0x600、0x700、0x800、0x900。这些虚拟地址指向地址空间前十页中每一页的第一个字节（每个虚拟地址的第一个十六进制数字就是页号）。

再假设除了虚拟页 3 以外，其他每一页都已经在内存中了。于是，这串内存引用会出现以下情况：命中、命中、命中、未命中、命中、命中、命中、命中、命中、命中。我们可以算出**命中率**（hit rate，即在内存中找到的引用所占的百分比）：90%，因为 10 次引用中有 9 次在内存中找到了。因此**未命中率**（miss rate）是 10%（$P_{\text{Miss}} = 0.1$）。一般来说，$P_{\text{Hit}} + P_{\text{Miss}} = 1.0$，即命中率加未命中率（miss rate）等于 100%。

要计算 AMAT，需要知道访问内存的开销和访问磁盘的开销。假设访问内存的开销（T_M）约为 100 纳秒，访问磁盘的开销（T_D）约为 10 毫秒，那么 AMAT 为：$100\text{ns} + 0.1 \cdot 10\text{ms}$，即 100ns + 1ms，也就是 1.0001 毫秒，大约 1 毫秒。如果命中率是 99.9%（$P_{\text{Miss}} = 0.001$），结果就大不一样了：AMAT 是 10.1 微秒，大约快了 100 倍。命中率越接近 100%，AMAT 就越接近 100 纳秒。

遗憾的是，从这个例子可以看出，在现代系统中磁盘访问的开销实在太高，哪怕未命中率（miss rate）很小，也会很快成为程序整体 AMAT 的主要部分。显然，我们必须尽可能避免未命中，否则程序就只能以磁盘的速度慢慢跑。办法之一就是精心设计一个聪明的策略，这正是我们接下来要做的。

22.2 最优置换策略

要更好地了解某个置换策略（replacement policy）表现如何，最好拿它和可能的最佳置换策略做个比较。其实，早在多年前 Belady 就提出了这样的**最优**（optimal）策略 [B66]（他最初称之为 MIN）。最优置换策略（replacement policy）产生的未命中总次数最少。Belady 证明，最优策略其实很简单（可惜很难实现！）：置换将来最晚才会被访问的那一页，这样产生的缓存未命中次数最少。

提示：和最优策略比较很有用

虽然最优策略作为实际策略并不实用，但在模拟或其他研究中，它作为比较的基准却极其有用。光说你那花哨的新算法命中率达到 80%，本身没什么意义；如果同时说明最优策略的命中率是 82%（因此你的新方法已经非常接近最优），结果就更有意义，也有了参照。所以，在你做的任何研究中，知道最优结果是多少，都能让你做出更好的比较：它能告诉你还有多大的改进空间，也能告诉你什么时候可以停止改进策略了，因为它已经足够接近理想状态 [AD03]。

最优策略背后的直觉应该不难理解。你可以这样想：如果必须扔掉某一页，为什么不扔掉要过最久才会用到的那一页呢?这样做，实际上就是认定缓存中其他所有页都比这一页重要。理由很简单：你会先用到其他那些页，然后才会用到它。

我们来跟踪一个简单的例子，看看最优策略会做出哪些决定。假设程序依次访问以下虚拟页：0、1、2、0、1、3、0、3、1、2、1。图 22.1 展示了最优策略的行为，这里假设缓存能放三页。

从图中可以看到以下过程。毫不奇怪，前三次访问都是未命中，因为缓存一开始是空的；这种未命中有时称为**冷启动未命中**（cold-start miss），或**强制性未命中**（compulsory miss）。接着我们再次引用页 0 和页 1，它们都在缓存中命中。然后，我们又遇到一次未命中（访问页 3），但这次缓存已经满了，必须进行置换！问题来了：该置换哪一页？用最优策略，我们会查看当前缓存中每一页（0、1、2）将来的情况，发现 0 几乎马上就会被访问，1 稍后会被访问，而 2 要过最久才会被访问。所以最优策略很容易做出选择：淘汰页 2，于是缓存中是页 0、1、3。接下来的三次引用都命中了，但随后访问页 2 时（它早就被淘汰了），又发生了一次未命中。这时最优策略再次查看缓存中每一页（0、1、3）将来的情况，发现只要不淘汰页 1（它马上就要被访问），就没问题。示例中淘汰的是页 3，其实淘汰页 0 也同样可以。最后，访问页 1 命中，这串访问序列就结束了。

我们还可以计算这个缓存的命中率：命中 6 次、未命中 5 次，命中率等于命中次数 /（命中次数 + 未命中次数），即 6 /（6 + 5），也就是 54.5%。也可以算出不计强制性未命中（compulsory miss）时的命中率（即每一页的第一次未命中不算），结果是 85.7%。

遗憾的是，正如我们之前在设计调度策略（scheduling policy）时看到的，未来通常是无法预知的；通用操作系统不可能实现最优策略[1]。因此，在设计真正能部署的实际策略时，我们要另想办法来决定淘汰哪一页。最优策略只用作比较的基准，让我们知道离“完美”还有多远。

访问	命中/未命中?	淘汰	访问后的缓存状态
0	未命中		0
1	未命中		0, 1
2	未命中		0, 1, 2
0	命中		0, 1, 2
1	命中		0, 1, 2
3	未命中	2	0, 1, 3
0	命中		0, 1, 3
3	命中		0, 1, 3
1	命中		0, 1, 3
2	未命中	3	0, 1, 2
1	命中		0, 1, 2

图 22.1：**最优策略的跟踪过程**

[1]如果你能做到，请告诉我们！我们可以一起发财。要不然，就一起像那些“发现”了冷核聚变的科学家那样，遭到众人的嘲笑和讥讽 [FP89]。

 © 2008–25, ARPACI-DUSSEAU

补充：缓存未命中的类型

在计算机体系结构领域，设计者发现，把未命中按类型分成三类有时很有用：强制性未命中（compulsory miss）、容量未命中（capacity miss）和冲突未命中（conflict miss），有时合称 **3C**（Three C's）[H87]。**强制性未命中**（compulsory miss，也叫**冷启动未命中**，cold-start miss [EF78]）是因为缓存一开始是空的，而这是对该数据项的第一次引用；相比之下，**容量未命中**（capacity miss）是因为缓存空间用完了，必须淘汰一项才能把新的一项放进缓存。第三类未命中（**冲突未命中**，conflict miss）出现在硬件中，原因是硬件缓存对数据项能放在哪里有限制，这和所谓的**组相联**（set-associativity）有关；操作系统的页缓存（page cache）不会出现这种未命中，因为这类缓存总是**全相联**（fully-associative）的，也就是说，页可以放在内存中的任何位置，没有限制。详见 H&P [HP06]。

22.3 一个简单的策略：FIFO

许多早期系统不去费力逼近最优，而是采用了非常简单的置换策略（replacement policy）。例如，有些系统采用 **FIFO**（先进先出，first-in, first-out）置换：页进入系统时，只是被放进一个队列（queue）；需要置换时，淘汰队列尾部的那一页（即“最先进入”的页）。FIFO 有一个很大的优点：实现起来非常简单。

我们来看看 FIFO 在示例引用序列上的表现（图 22.2，第 4 页）。跟踪过程同样以页 0、1、2 的三次强制性未命中（compulsory miss）开始，然后页 0 和页 1 都命中了。接着引用页 3，未命中；用 FIFO 做置换决定很简单：选最先进来的那一页（图中的缓存状态按 FIFO 顺序排列，最先进入的页在最左边），也就是页 0。不巧的是，下一次访问的正是页 0，于是又一次未命中，又要置换（这次换掉页 1）。之后页 3 命中，但页 1 和页 2 都未命中，最后页 1 命中。

访问	命中/未命中?	淘汰	访问后的缓存状态
0	未命中		先入→ 0
1	未命中		先入→ 0, 1
2	未命中		先入→ 0, 1, 2
0	命中		先入→ 0, 1, 2
1	命中		先入→ 0, 1, 2
3	未命中	0	先入→ 1, 2, 3
0	未命中	1	先入→ 2, 3, 0
3	命中		先入→ 2, 3, 0
1	未命中	2	先入→ 3, 0, 1
2	未命中	3	先入→ 0, 1, 2
1	命中		先入→ 0, 1, 2

图 22.2：**FIFO 策略的跟踪过程**

补充：Belady 异常

Belady（就是提出最优策略的那位）和同事发现了一个有趣的引用序列，它的表现有点出人意料 [BNS69]。这个内存引用序列是：1、2、3、4、1、2、5、1、2、3、4、5。他们研究的置换策略（replacement policy）是 FIFO。有趣的地方在于：缓存大小从 3 页增加到 4 页时，缓存命中率会怎样变化。

一般来说，你会预期缓存变大时命中率会上升（变好）。但在这个例子里，用 FIFO 时命中率反而变差了！你可以自己算一算命中和未命中的次数。这种奇怪的现象通常称为 **Belady 异常**（Belady's Anomaly），这让他的合著者们很是郁闷。

另一些策略，比如 LRU，就不会出现这个问题。你能猜到为什么吗？原来，LRU 具有所谓的 **stack property** [M+70]。对具有这种属性的算法来说，大小为 $N+1$ 的缓存自然包含了大小为 N 的缓存中的内容。因此，增大缓存时，命中率要么不变，要么提高。FIFO 和随机策略（还有其他一些策略）显然不满足 stack property，因此可能出现这种异常行为。

和最优策略相比，FIFO 明显差得多：命中率只有 36.4%，不计强制性未命中（compulsory miss）也只有 57.1%。FIFO 根本无法判断各个块（block）的重要程度：尽管页 0 已经被访问过好几次，FIFO 还是把它踢了出去，仅仅因为它是最先被调入内存的那一页。

22.4 另一个简单的策略：随机

另一种类似的置换策略（replacement policy）是随机（Random）策略，它在内存压力（memory pressure）下只是随机挑一页来置换。随机策略的特点和 FIFO 类似：实现简单，但在挑选要淘汰的块时，并不费心去做得聪明些。我们来看看随机策略在我们那个著名的示例引用序列上表现如何（见图 22.3）。

访问	命中/未命中?	淘汰	访问后的缓存状态
0	未命中		0
1	未命中		0, 1
2	未命中		0, 1, 2
0	命中		0, 1, 2
1	命中		0, 1, 2
3	未命中	0	1, 2, 3
0	未命中	1	2, 3, 0
3	命中		2, 3, 0
1	未命中	3	2, 0, 1
2	命中		2, 0, 1
1	命中		2, 0, 1

图 22.3：**随机策略的跟踪过程**

 © 2008–25, ARPACI-DUSSEAU

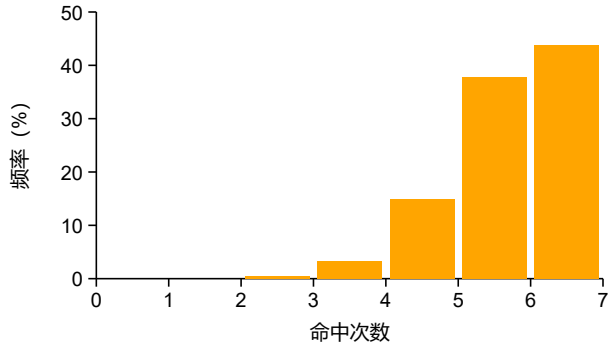


图 22.4：**随机策略在 10,000 次试验中的表现**

当然，随机策略的表现完全取决于它挑选时运气好坏。在上面的例子中，随机策略比 FIFO 稍好一点，比最优策略稍差一点。实际上，我们可以把随机策略的实验跑上几千次，看看它总体上表现如何。图 22.4 展示了随机策略在 10,000 次试验中的命中次数，每次试验使用不同的随机种子。可以看到，有时候（略多于 40% 的情况下），随机策略和最优策略一样好，在这个示例序列上命中 6 次；有时候它又差得多，只命中 2 次甚至更少。随机策略表现如何，全凭手气。

22.5 利用历史：LRU

遗憾的是，像 FIFO 或随机这样简单的策略，大概都有一个共同的问题：可能会把重要的页踢出去，也就是马上又要被引用的页。FIFO 踢出的是最先调入的页；如果这一页上恰好有重要的代码或数据结构（data structure），它照样会被扔掉，尽管很快又得把它调回来。因此，FIFO、随机以及类似的策略不太可能接近最优，我们需要更聪明的办法。

就像在调度策略中做过的那样，为了更好地预测未来，我们再一次借助过去，以历史为指导。例如，如果程序最近访问过某一页，那么它很可能在不久的将来再次访问这一页。

页面置换策略（page-replacement policy）能利用的一种历史信息是**频率**（frequency）：如果一页被访问了很多次，也许就不应该置换它，因为它显然有一定价值。更常用的是页的另一个属性，即访问的**近期性**（recency）：一页最近一次被访问的时间越近，也许就越有可能再次被访问。

这一类策略基于人们所说的**局部性原理**（principle of locality）[D70]，这条原理其实只是对程序及其行为的一种观察。简单地说，它指的是：程序往往会非常频繁地访问某些代码序列（例如在循环中）和数据结构（例如循环访问的数组）；因此，我们应该设法利用历史来判断哪些页是重要的，到了要淘汰的时候，把这些页留在内存中。

于是，一类基于历史的简单算法就诞生了。**最不经常使用**（Least-Frequently-Used，**LFU**）策略在必须淘汰时，置换使用频率最低的那一页。类似地，**最近最少使用**（Least-Recently-Used，**LRU**）策略置换最近最少使用的那一页。这些算法很好记：知道了名字，就确切地知道它做什么。对一个名字来说，这是再好不过的特性。

 © 2008–25, ARPACI-DUSSEAU

访问	命中/未命中?	淘汰	访问后的缓存状态
0	未命中		LRU→ 0
1	未命中		LRU→ 0, 1
2	未命中		LRU→ 0, 1, 2
0	命中		LRU→ 1, 2, 0
1	命中		LRU→ 2, 0, 1
3	未命中	2	LRU→ 0, 1, 3
0	命中		LRU→ 1, 3, 0
3	命中		LRU→ 1, 0, 3
1	命中		LRU→ 0, 3, 1
2	未命中	0	LRU→ 3, 1, 2
1	命中		LRU→ 3, 2, 1

图 22.5：**LRU 策略的跟踪过程**

为了更好地理解 LRU，我们来看看 LRU 在示例引用序列上的表现。图 22.5（第 7 页）展示了结果。从图中可以看到，LRU 如何利用历史信息，做得比随机或 FIFO 这样的无状态（stateless）策略更好。在这个例子中，LRU 第一次需要置换时淘汰了页 2，因为 0 和 1 都是在它之后才被访问的。接着它置换了页 0，因为 1 和 3 的访问都比它更近。在这两次置换中，LRU 基于历史做出的决定都是正确的，因此接下来的引用都命中了。所以，在我们的例子中，LRU 做到了最好，性能和最优策略一样[2]。

我们还应该指出，这些算法也有与之相反的版本：**最经常使用**（Most-Frequently-Used，**MFU**）和**最近最常使用**（Most-Recently-Used，**MRU**）。在大多数情况下（不是所有情况!），这些策略效果都不好，因为它们非但不利用大多数程序表现出的局部性，反而对它视而不见。

补充：局部性的类型

程序通常会表现出两种局部性（locality）。第一种叫**空间局部性**（spatial locality）：如果页 P 被访问了，那么它周围的页（比如 P − 1 或 P + 1）很可能也会被访问。第二种叫**时间局部性**（temporal locality）：最近被访问过的页，很可能在不久的将来再次被访问。硬件系统的缓存层次结构（cache hierarchy）在很大程度上建立在“存在这些局部性”这一假设之上：硬件部署了多级指令缓存、数据缓存和地址转换缓存（address-translation cache），好让程序在存在这种局部性时跑得更快。

当然，人们常说的**局部性原理**（principle of locality），并不是所有程序都必须遵守的铁律。实际上，有些程序访问内存（或磁盘）的方式相当随机，在访问序列中几乎没有或完全没有局部性。因此，虽然设计任何缓存（无论硬件还是软件）时都应该把局部性放在心上，但它并不能保证成功。它只是一种启发式方法，在计算机系统设计中往往很管用。

[2]好吧，结果是我们凑出来的。但有时候，为了说明一个观点，凑结果是必要的。

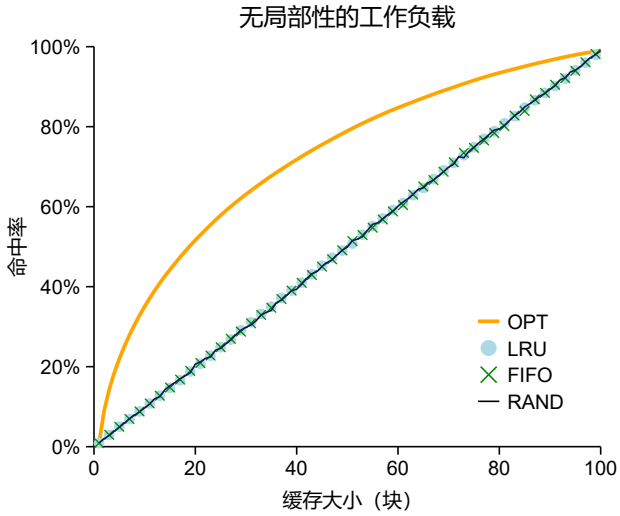


图 22.6：**无局部性的工作负载**

22.6 工作负载示例

我们再看几个例子，以便更好地理解其中一些策略的行为。这里我们研究更复杂的**工作负载**（workload），而不是很短的访问序列。不过，即使是这些工作负载，也做了大量简化；更好的研究应该使用真实应用程序的访问记录。

第一个工作负载没有局部性，也就是说，每次引用的都是被访问页集合中的一个随机页。在这个简单的例子中，工作负载一共涉及 100 个不同的页，每次随机选择下一个要引用的页，总计访问 10,000 次。在实验中，我们让缓存大小从很小（1 页）一直变到足以容纳所有不同的页（100 页），以观察各个策略在不同缓存大小下的表现。

图 22.6 画出了最优、LRU、随机和 FIFO 策略的实验结果。图的 y 轴是每个策略达到的命中率；x 轴是按上面所说变化的缓存大小。

从图中可以得出几个结论。第一，当工作负载没有局部性时，用哪种实际可行的策略都差不多；LRU、FIFO 和随机策略的表现完全一样，命中率完全由缓存大小决定。第二，当缓存大到足以容纳整个工作负载时，用哪种策略也都无所谓；只要被引用的所有块都放得进缓存，所有策略（甚至随机策略）的命中率都会趋近 100%。最后，可以看到最优策略明显好于这些实际可行的策略；要是真能偷看未来，置换的效果会好得多。

我们接下来研究的工作负载叫作“80-20”工作负载，它表现出局部性：80% 的引用落在 20% 的页上（“热”页）；其余 20% 的引用落在另外 80% 的页上（“冷”页）。这个工作负载同样一共有 100 个不同的页；因此，大部分时间引用的是“热”页，其余时间引用的是“冷”页。图 22.7（第 9 页）展示了各个策略在这个工作负载下的表现。

 © 2008–25, ARPACI-DUSSEAU

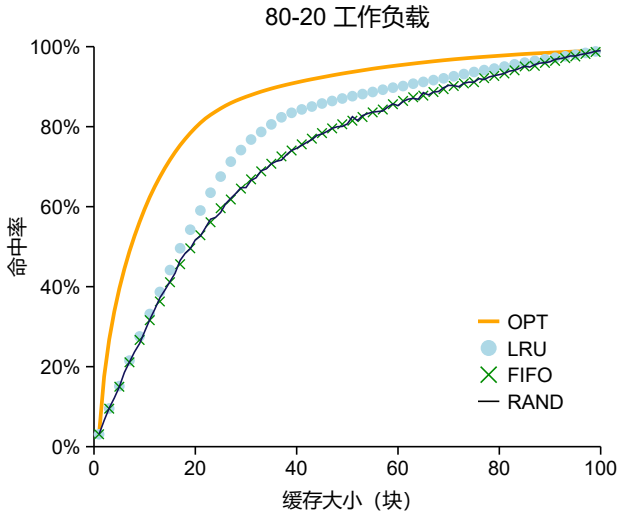


图 22.7：**80-20 工作负载**

从图中可以看出，虽然随机策略和 FIFO 的表现都还不错，但 LRU 更好，因为它更有可能留住热页（hot page）；这些页过去经常被引用，很可能在不久的将来再次被引用。最优策略又一次表现得更好，这说明 LRU 的历史信息并不完美。

你现在可能想问：LRU 相对于随机和 FIFO 的改进，真的有那么重要吗？答案和往常一样：“看情况。”如果每次未命中的代价都很高（这并不少见），那么即使命中率只提高一点点，也就是未命中率（miss rate）只降低一点点，也会给性能带来巨大的差别。如果未命中的代价没那么高，那么 LRU 可能带来的好处自然也就没那么重要了。

我们再看最后一个工作负载。我们把它叫作“循环顺序”工作负载：按顺序引用 50 个页，从 0 开始，然后是 1、……，一直到页 49，然后循环回来重复这些访问，对这 50 个不同的页总共访问 10,000 次。最后这张图（图 22.8）展示了各个策略在这个工作负载下的表现。

这种工作负载在许多应用程序中都很常见（包括数据库这类重要的商业应用 [CD85]），它对 LRU 和 FIFO 来说都是最坏情况。在循环顺序工作负载下，这两种算法会踢出较老的页；可惜，由于工作负载是循环的，这些较老的页反而会比策略想留在缓存里的那些页更早被访问。实际上，对 50 个页的循环顺序工作负载来说，即使缓存能放下 49 页，命中率也是 0%。有意思的是，随机策略的表现明显更好，虽然离最优还有差距，但至少命中率不为零。看来随机策略也有一些不错的特性，其中之一就是不会在极端情况下出现怪异的行为。

22.7 实现基于历史的算法

可以看到，像 LRU 这样的算法通常比 FIFO 或随机这样较简单的策略做得更好，后者可能会扔掉重要的页。遗憾的是，基于历史的策略给我们带来了一个新的挑战：怎么实现它们？

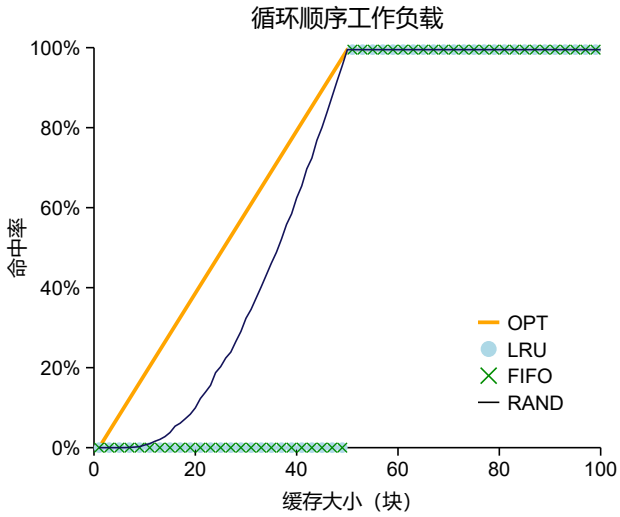


图 22.8：**循环工作负载**

以 LRU 为例。要完美地实现它，需要做很多工作。具体来说，每次访问页时（即每次内存访问，无论是取指令还是加载、存储），我们都必须更新某个数据结构，把这一页移到链表（linked list）的最前面（即 MRU 一侧）。相比之下，FIFO 只有在淘汰（evict）一页（移除最先进入的页）或者向链表中加入新页（new page，加到最后进入的一侧）时，才会访问它的页链表。为了记录哪些页是最近最少使用（Least-Recently-Used）的、哪些是最近最常使用（Most-Recently-Used）的，系统必须在每次内存引用时都做一些记录工作。显然，如果不格外小心，这种记录工作可能会大大降低性能。

加快速度的一个办法是加一点硬件支持。例如，机器可以在每次访问页时更新内存中的一个时间字段（比如，这个字段可以放在每个进程的页表（page table）中，或者干脆放在内存中某个单独的数组里，系统中每个物理页（physical page）对应一项）。这样，一页被访问时，硬件就会把它的时间字段设为当前时间。然后，置换页时，操作系统只需扫描系统中所有的时间字段，就能找到最近最少使用（Least-Recently-Used）的页。

可惜，系统中的页越来越多时，仅仅为了找到绝对意义上最近最少使用（Least-Recently-Used）的那一页，就去扫描一个巨大的时间数组，代价高得让人无法接受。设想一台有 4 GB 内存的现代机器，内存被切成 4 KB 的页。这台机器有 100 万个页，因此即使以现代 CPU 的速度，找到 LRU 页也要花很长时间。这就引出了一个问题：我们真的需要找到绝对最老的那一页来置换吗？用一个近似的结果能不能凑合？

关键问题：如何实现 LRU 置换策略
既然实现完美的 LRU 代价很高，我们能否用某种方式近似它，同时仍然获得想要的效果？

 © 2008–25, ARPACI-DUSSEAU

22.8 近似 LRU

答案是肯定的：从计算开销的角度看，近似 LRU 更为可行，而且许多现代系统实际上也正是这么做的。这个想法需要一点硬件支持，即**访问位**（use bit，有时也叫 **reference bit**）。最早的访问位出现在第一个支持分页（paging）的系统，也就是 Atlas 单级存储（one-level store）中 [KE+62]。系统中每一页都有一个访问位，这些访问位存放在内存中的某个地方（比如可以放在每个进程的页表中，或者干脆放在某处的一个数组里）。每当一页被引用（即被读或写）时，硬件就把它的访问位设为 1。不过，硬件从不清除这个位（即把它设为 0），这是操作系统的职责。

操作系统如何利用访问位来近似 LRU 呢？方法可能有很多，**时钟算法**（clock algorithm）[C69] 就是其中一种简单的做法。想象系统中所有的页排成一个环形链表。一根**时钟指针**（clock hand）一开始指向某一页（具体是哪一页并不重要）。需要置换时，操作系统检查当前指向的页 P 的访问位是 1 还是 0。如果是 1，说明页 P 最近被使用过，因此不是置换的好人选。于是把 P 的访问位设为 0（清除），时钟指针（clock hand）前进到下一页（P + 1）。算法一直这样进行下去，直到找到一个访问位为 0 的页，这说明这一页最近没有被使用过（最坏的情况则是所有页最近都被使用过，这时我们已经把所有页搜索了一遍，清除了全部访问位）。

注意，这并不是利用访问位来近似 LRU 的唯一方法。实际上，只要定期清除访问位，再根据各页的访问位是 1 还是 0 来决定置换哪一页，任何这样的方法都可以。Corbato 的时钟算法（clock algorithm）只是早期取得了一定成功的方法之一，它有一个很好的特性：不需要反复扫描整个内存来寻找未被使用的页。

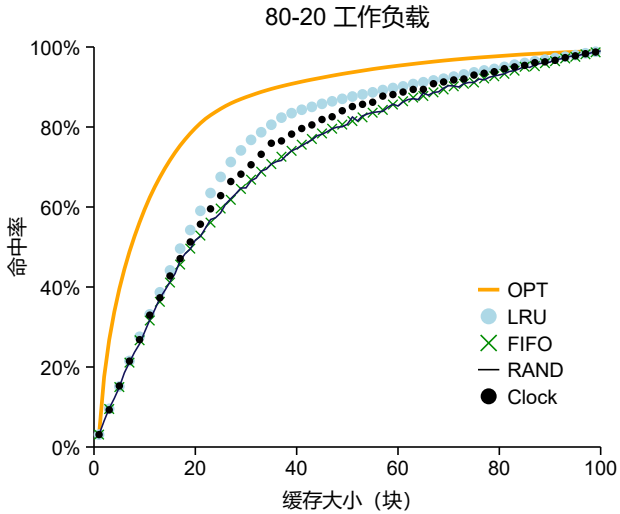


图 22.9：**80-20 工作负载（加入时钟算法）**

图 22.9 展示了时钟算法（clock algorithm）某个变体的表现。这个变体在置换时随机扫描各页：遇到访问位为 1 的页，就清除这个位（即设为 0）；找到访问位为 0 的页，就选它作为淘汰对象。可以看到，虽然它比不上完美的 LRU，但比完全不考虑历史的方法要好。

22.9 考虑脏页

对时钟算法（clock algorithm）常做的一个小修改（同样最早由 Corbato 提出 [C69]），是额外考虑一页在内存中是否被修改过。这样做的原因是：如果一页已经被**修改**（modified）过，因而是**脏**（dirty）的，那么要淘汰它就必须先把它写回磁盘，这代价很高。如果它没有被修改过，因而是**干净**（clean）的，淘汰它就没有任何开销；对应的物理帧（physical frame）可以直接另作他用，不需要额外的 I/O。因此，有些虚拟内存系统更倾向于淘汰干净页（clean page）而不是脏页（dirty page）。

为了支持这种行为，硬件应该提供一个**修改位**（modified bit，又称**脏位**，dirty bit）。每当一页被写入时，这个位就会被置位，因此可以把它纳入页面置换（page replacement）算法中。例如，可以修改时钟算法（clock algorithm），让它优先寻找既未被使用又干净的页来淘汰；如果找不到，再找未被使用但脏的页，依此类推。

22.10 其他虚拟内存策略

页面置换（page replacement）并不是虚拟内存子系统采用的唯一策略（尽管它可能是最重要的）。例如，操作系统还必须决定什么时候把一页调入内存。这个策略有时称为**页面选择**（page selection）策略，Denning 就是这么叫它的 [D70]。在这方面，操作系统有几种不同的做法可选。

对大多数页，操作系统只是采用**按需分页**（demand paging），也就是说，在页被访问时才把它调入内存，可以说是“按需”调入。当然，操作系统也可以猜测某一页马上就要被用到，于是提前把它调进来；这种行为称为**预取**（prefetching），只有在有相当把握猜中时才应该这么做。例如，有些系统会假设：如果代码页 P 被调入了内存，那么代码页 P + 1 很可能马上就会被访问，因此也应该一并调入内存。

另一个策略决定操作系统如何把页写到磁盘上。当然，可以一次只写一页；不过，许多系统会先在内存中把一批待写的页收集起来，然后一次（更高效地）写到磁盘上。这种做法通常称为写操作的**聚集**（clustering），或者干脆叫**成组**（grouping）。它之所以有效，是因为磁盘驱动器（disk drive）有这样的特性：执行一次大的写操作，比执行许多次小的写操作效率更高。

22.11 抖动

在结束本章之前，我们来讨论最后一个问题：如果内存根本不够用，也就是正在运行的这组进程对内存的需求超出了可用的物理内存，操作系统该怎么办？在这种情况下，系统会不停地换页，这种状况有时称为**抖动**（thrashing）[D70]。

一些早期的操作系统有一套相当精巧的机制，能在抖动（thrashing）发生时检测到它并加以应对。例如，对于一组进程，系统可以决定不运行其中的一部分，希望剩下那些进程的**工作集**（working set，即它们正在活跃使用的页）能够放进内存，让这些进程得以继续推进。这种方法一般称为**准入控制**（admission control），它的道理是：有时候少做一些但做好，比一下子什么都做却什么都做不好要强。这种情况我们在现实生活中也常常遇到，在现代计算机系统中同样如此（很遗憾）。

当前的一些系统对内存过载采取了更加严厉的做法。例如，某些版本的 Linux 在内存不够用时会运行**内存耗尽杀手**（out-of-memory killer，即 OOM killer）；这个守护进程（daemon）会选出一个占用大量内存的进程并杀掉它，以相当粗暴的方式减少内存占用。虽然这种方法能成功缓解内存压力（memory pressure），但也可能带来问题，比如它要是杀掉了 X 服务器，所有需要显示界面的应用程序就都没法用了。

22.12 小结

我们已经介绍了一些页面置换策略（page-replacement policy）以及其他策略，它们是所有现代操作系统虚拟内存子系统的一部分。现代系统会对时钟算法（clock algorithm）这类简单直接的 LRU 近似算法做一些调整；例如，**抗扫描**（scan resistance）是 ARC [MM03] 等许多现代算法的重要组成部分。抗扫描算法（scan-resistant algorithm）通常和 LRU 类似，但会尽量避开 LRU 的最坏情况，也就是我们在循环顺序工作负载中看到的那种情况。因此，页面置换（page replacement）算法仍在不断演进。

过去很多年里，置换算法（replacement algorithm）的重要性有所下降，因为内存访问时间和磁盘访问时间相差实在太大。具体来说，由于换页到磁盘的代价非常高，频繁换页的开销让人无法承受；简单地说，无论你的置换算法（replacement algorithm）多好，只要频繁置换，系统就会慢得无法忍受。因此，最好的解决办法很简单（虽然不怎么高明）：买更多内存。

然而，近年来基于闪存（flash）的 SSD 等速度快得多的存储设备（storage device）不断推陈出新，又一次改变了这些性能比例，页面置换（page replacement）算法也随之迎来复兴。这方面的最新工作见 [SS10, W+21]。

参考文献

[AD03] "Run-Time Adaptation in River" by Remzi H. Arpaci-Dusseau. ACM TOCS, 21:1, February 2003. *A summary of one of the authors' dissertation work on a system named River, where he learned that comparison against the ideal is an important technique for system designers.*

[B66] "A Study of Replacement Algorithms for Virtual-Storage Computer" by Laszlo A. Belady. IBM Systems Journal 5(2): 78-101, 1966. *The paper that introduces the simple way to compute the optimal behavior of a policy (the MIN algorithm).*

[BNS69] "An Anomaly in Space-time Characteristics of Certain Programs Running in a Paging Machine" by L. A. Belady, R. A. Nelson, G. S. Shedler. Communications of the ACM, 12:6, June 1969. *Introduction of the little sequence of memory references known as Belady's Anomaly. How do Nelson and Shedler feel about this name, we wonder?*

[CD85] "An Evaluation of Buffer Management Strategies for Relational Database Systems" by Hong-Tai Chou, David J. DeWitt. VLDB '85, Stockholm, Sweden, August 1985. *A famous database paper on the different buffering strategies you should use under a number of common database access patterns. The more general lesson: if you know something about a workload, you can tailor policies to do better than the general-purpose ones usually found in the OS.*

[C69] "A Paging Experiment with the Multics System" by F.J. Corbato. Included in a Festschrift published in honor of Prof. P.M. Morse. MIT Press, Cambridge, MA, 1969. *The original (and hard to find!) reference to the clock algorithm, though not the first usage of a use bit. Thanks to H. Balakrishnan of MIT for digging up this paper for us.*

[D70] "Virtual Memory" by Peter J. Denning. Computing Surveys, Vol. 2, No. 3, September 1970. *Denning's early and famous survey on virtual memory systems.*

[EF78] "Cold-start vs. Warm-start Miss Ratios" by Malcolm C. Easton, Ronald Fagin. Communications of the ACM, 21:10, October 1978. *A good discussion of cold- vs. warm-start misses.*

[FP89] "Electrochemically Induced Nuclear Fusion of Deuterium" by Martin Fleischmann, Stanley Pons. Journal of Electroanalytical Chemistry, Volume 26, Number 2, Part 1, April, 1989. *The famous paper that would have revolutionized the world in providing an easy way to generate nearly-infinite power from jars of water with a little metal in them. Unfortunately, the results published (and widely publicized) by Pons and Fleischmann were impossible to reproduce, and thus these two well-meaning scientists were discredited (and certainly, mocked). The only guy really happy about this result was Marvin Hawkins, whose name was left off this paper even though he participated in the work, thus avoiding association with one of the biggest scientific goofs of the 20th century.*

[HP06] "Computer Architecture: A Quantitative Approach" by John Hennessy and David Patterson. Morgan-Kaufmann, 2006. *A marvelous book about computer architecture. Read it!*

[H87] "Aspects of Cache Memory and Instruction Buffer Performance" by Mark D. Hill. Ph.D. Dissertation, U.C. Berkeley, 1987. *Mark Hill, in his dissertation work, introduced the Three C's, which later gained wide popularity with its inclusion in H&P [HP06]. The quote from therein: "I have found it useful to partition misses … into three components intuitively based on the cause of the misses (page 49)."*

[KE+62] "One-level Storage System" by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Trans. EC-11:2, 1962. *Although Atlas had a use bit, it only had a very small number of pages, and thus the scanning of the use bits in large memories was not a problem the authors solved.*

[M+70] "Evaluation Techniques for Storage Hierarchies" by R. L. Mattson, J. Gecsei, D. R. Slutz, I. L. Traiger. IBM Systems Journal, Volume 9:2, 1970. *A paper that is mostly about how to simulate cache hierarchies efficiently; certainly a classic in that regard, as well for its excellent discussion of some of the properties of various replacement algorithms. Can you figure out why the stack property might be useful for simulating a lot of different-sized caches at once?*

[MM03] "ARC: A Self-Tuning, Low Overhead Replacement Cache" by Nimrod Megiddo and Dharmendra S. Modha. FAST 2003, February 2003, San Jose, California. *An excellent modern paper about replacement algorithms, which includes a new policy, ARC, that is now used in some systems. Recognized in 2014 as a "Test of Time" award winner by the storage systems community at the FAST '14 conference.*

[SS10] "FlashVM: Virtual Memory Management on Flash" by Mohit Saxena, Michael M. Swift. USENIX ATC '10, June, 2010, Boston, MA. *An early, excellent paper by our colleagues at U. Wisconsin about how to use Flash for paging. One interesting twist is how the system has to take **wearout**, an intrinsic property of Flash-based devices, into account. Read more about Flash-based SSDs later in this book if you are interested.*

 © 2008–25, ARPACI-DUSSEAU

[W+21] “The Storage Hierarchy is Not a Hierarchy: Optimizing Caching on Modern Storage Devices with Orthus” by Kan Wu, Zhihan Guo, Guanzhou Hu, Kaiwei Tu, Ramnatthan Alagappan, Rathijit Sen, Kwanghyun Park, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau. FAST ’21, held virtually — thanks, COVID-19. *Our own work on a caching approach on modern devices; is it directly related to page replacement? Perhaps not. But it is a fun paper.*

作业（模拟）

模拟器 `paging-policy.py` 可以让你试验各种页面置换策略。详情请见 README。

问题

1. 用以下选项生成随机地址：`-s 0 -n 10`、`-s 1 -n 10` 和 `-s 2 -n 10`。把策略依次改为 FIFO、LRU、OPT。判断这些地址序列中的每次访问是命中还是未命中。
2. 对于大小为 5 的缓存，分别为以下策略生成最坏情况的地址引用序列：FIFO、LRU 和 MRU（最坏情况的引用序列会导致尽可能多的未命中）。对于这些最坏情况的引用序列，缓存要大多少，性能才会显著提高并接近 OPT？
3. 生成一个随机的访问序列（即用 Python 写一个脚本，输出随机地址，然后把这些地址输入模拟器）。你预计不同的策略在这样的序列上会表现如何？
4. 现在生成一个具有一定局部性的访问序列。怎样才能生成这样的序列？LRU 在它上面表现如何？LRU 比 RAND 好多少？CLOCK 表现如何？如果 CLOCK 使用不同数量的时钟位，结果又如何？
5. 用 valgrind 这样的程序对一个真实应用程序进行插桩，生成虚拟页引用序列。例如，运行 `valgrind --tool=lackey --trace-mem=yes ls` 会输出一份近乎完整的访问记录，包含程序 `ls` 的每一次指令引用和数据引用。要让它能用于上面的模拟器，你得先把每个虚拟内存引用转换成虚拟页号（virtual page number）引用（方法是屏蔽掉偏移量，再把剩下的位右移）。对你的应用程序的访问序列来说，缓存要多大才能满足大部分请求？画一张图，展示缓存逐渐增大时它的工作集（working set）情况。

 © 2008–25, ARPACI-DUSSEAU